

OFFLINEVAULT

Security Assessment - Portfolio Summary

Assessment type: Authorized local testing
Scope: Authentication, encryption, lock behavior, storage, lifecycle and basic input testing
Data: Dummy credentials and test entries only
Version: 1.0

At a glance

3	3	3	2
Confirmed security findings	Important controls passed	High/medium priority issues	Tests not fully conclusive

Executive summary

OfflineVault uses a real encryption design and several useful security controls, but the assessment found weaknesses in the **lock workflow** and **master-password protection**. The most important issue is that the visible **Lock** button did not actually lock the vault during testing. Very weak master passwords were also accepted, including a one-character password. Repeated wrong unlock attempts showed no observed delay or lockout in the first 30 attempts.

Key findings

OFFV-01 - Lock button does not actually lock the vault

After pressing Lock, the vault remained usable. The dummy entry could still be opened and its password revealed without entering the master password again. This is the most important issue because the UI can give a false sense of security.

OFFV-02 - Extremely weak master passwords are accepted

A one-character master password was accepted and unlocked the vault. A dummy vault protected by 123456 was also recovered using a small offline dictionary. The encryption itself was not broken; the weakness is the lack of password-strength enforcement.

OFFV-03 - No lockout/delay observed after 30 failed attempts

Thirty incorrect unlock attempts were tested with no observed temporary lockout, increasing delay or button disable. The test proves the behavior for those first 30 attempts, not for unlimited attempts.

What worked

Control	Result	Observation
Encryption	PASS	SCrypt + AES-GCM protected the vault; correct passwords decrypted it and incorrect passwords did not.
Background protection	PASS	Sending the app to the background and reopening required the master password again.
Screenshot protection	PASS	Secure window flag was observed; recent-app preview did not expose the password screen.
Password masking	PASS	Password remained masked in Edit mode and normal copy was not available.
Deletion	PASS	After deleting the dummy entry, the current decrypted vault contained an empty entries list.
Uninstall	PASS	After uninstall/reinstall, the previous local vault did not survive.

How the assessment was performed

Testing was performed against the local app/emulator using authorized access and dummy values. The assessment combined hands-on behavior testing with static review of the APK. Tests were deliberately kept practical and lightweight.

Area	What was tested
Authentication	Wrong-password attempts, one-character password, SQL-style input
Local protection	Vault file inspection, plaintext search, decryption verification
Encryption	Verified vault format, SCrypt parameters, AES-GCM decryption with correct/incorrect passwords
Lifecycle	Background/reopen behavior and state clearing
UI security	Lock button, password masking, Show Password, recent-app/screenshot behavior
Data handling	Clipboard check, entry deletion, uninstall/reinstall

Evidence highlights

Weak-password test	Master password 1 was accepted. A separate dummy vault protected by 123456 was recovered by a small local dictionary.
Brute-force UI test	30 incorrect unlock attempts produced the normal error without an observed lockout or progressive delay.
Lock test	After pressing the visible Lock control, the vault and entry actions remained available, including Show Password.
Background test	Using the emulator Home action and reopening the app returned to the unlock screen.
Deletion test	The post-delete vault was decrypted successfully and returned {"version":1,"entries":[]}
SQL-style test	The classic payload ' OR '1'='1' -- did not bypass the vault. It behaved like an incorrect master password.

Recommended fixes - priority order

- **1. Fix the Lock button:** pressing Lock should immediately clear the unlocked vault state and force re-authentication for every secret-view action.
- **2. Enforce a strong master password:** reject very short and common passwords and explain the rule clearly to the user.
- **3. Add failed-attempt protection:** use progressive delays or temporary lockout after repeated failures.
- **4. Verify the production build:** ensure debugging is disabled in the release version and review exported components.
- **5. Validate backups:** repeat backup/restore testing on a backup-enabled device because this assessment could not confirm backup behavior.

Important: The backup question was not marked as a vulnerability because the test emulator reported that its Backup Manager was disabled. Also, the exact internal callback behind the Lock-button failure was not proven statically; the runtime behavior was reproduced.

Portfolio takeaway

This assessment demonstrates practical security testing: reproduce a behavior, collect evidence, distinguish confirmed issues from limitations, and document both weaknesses and controls that work. The goal was not simply to find bugs, but to understand the app's security posture and provide actionable fixes.

Assessment note: Authorized testing only. Dummy credentials/data were used throughout the assessment. Severity labels are practical engineering judgments for this password-vault context and are not a formal CVSS score.